

Análise de Código Java

SlidingWindowMax — Máximo em Janela Deslizante

1. Objetivo Geral

O código implementa o algoritmo de **janela deslizante com deque monotónico** para encontrar, de forma eficiente, o valor máximo dentro de cada janela de tamanho k que percorre um array de inteiros. O método `maxSlidingWindow` retorna um array de resultados com todos os máximos de cada posição da janela.

2. Lógica e Algoritmo

Percorre-se o array com um índice i mantendo um **deque de índices** ordenado de modo a que `nums[deque.front]` seja sempre o maior elemento da janela actual. A cada iteração: (1) remove-se da frente índices que saíram da janela ($i - k + 1$); (2) remove-se do fim índices cujos valores são menores que `nums[i]`, preservando a ordem decrescente; (3) adiciona-se i no fim; (4) a partir de $i \geq k - 1$, o elemento na frente do deque é o máximo da janela actual.

3. Conceitos e Funcionalidades Java

O código utiliza a interface `Deque<Integer>` com implementação `ArrayDeque`, que oferece operações $O(1)$ nas duas extremidades. Os métodos `peekFirst`, `pollFirst`, `peekLast`, `pollLast` e `addLast` são centrais para a lógica do deque monotónico. A classe `Arrays` é usada no `main` para imprimir os resultados.

4. Complexidade Temporal e Espacial

A complexidade temporal é $O(n)$: cada índice é inserido e removido do deque no máximo uma vez. A complexidade espacial é $O(k)$ para o deque e $O(n - k + 1)$ para o array de resultados, resultando em $O(n)$ no total.

5. Decisões de Design e Limitações

A escolha do deque monotónico evita a abordagem ingénua de $O(n \cdot k)$, tornando o algoritmo óptimo para grandes inputs. Uma limitação relevante é a ausência de validação de entrada: o código não verifica se $k \leq 0$, se $k > n$, ou se o array é nulo, o que pode gerar `NegativeArraySizeException` ou `NullPointerException` em cenários reais.

Código Analisado

```
public static int[] maxSlidingWindow(int[] nums, int k) {
    int n = nums.length;
    int[] result = new int[n - k + 1];
    Deque<Integer> deque = new ArrayDeque<>();

    for (int i = 0; i < n; i++) {
```

```
        while (!deque.isEmpty() && deque.peekFirst() < i - k + 1)
            deque.pollFirst();
        while (!deque.isEmpty() && nums[deque.peekLast()] < nums[i])
            deque.pollLast();
        deque.addLast(i);
        if (i >= k - 1)
            result[i - k + 1] = nums[deque.peekFirst()];
    }
    return result;
}
```